

可靠信道编码

理论课程

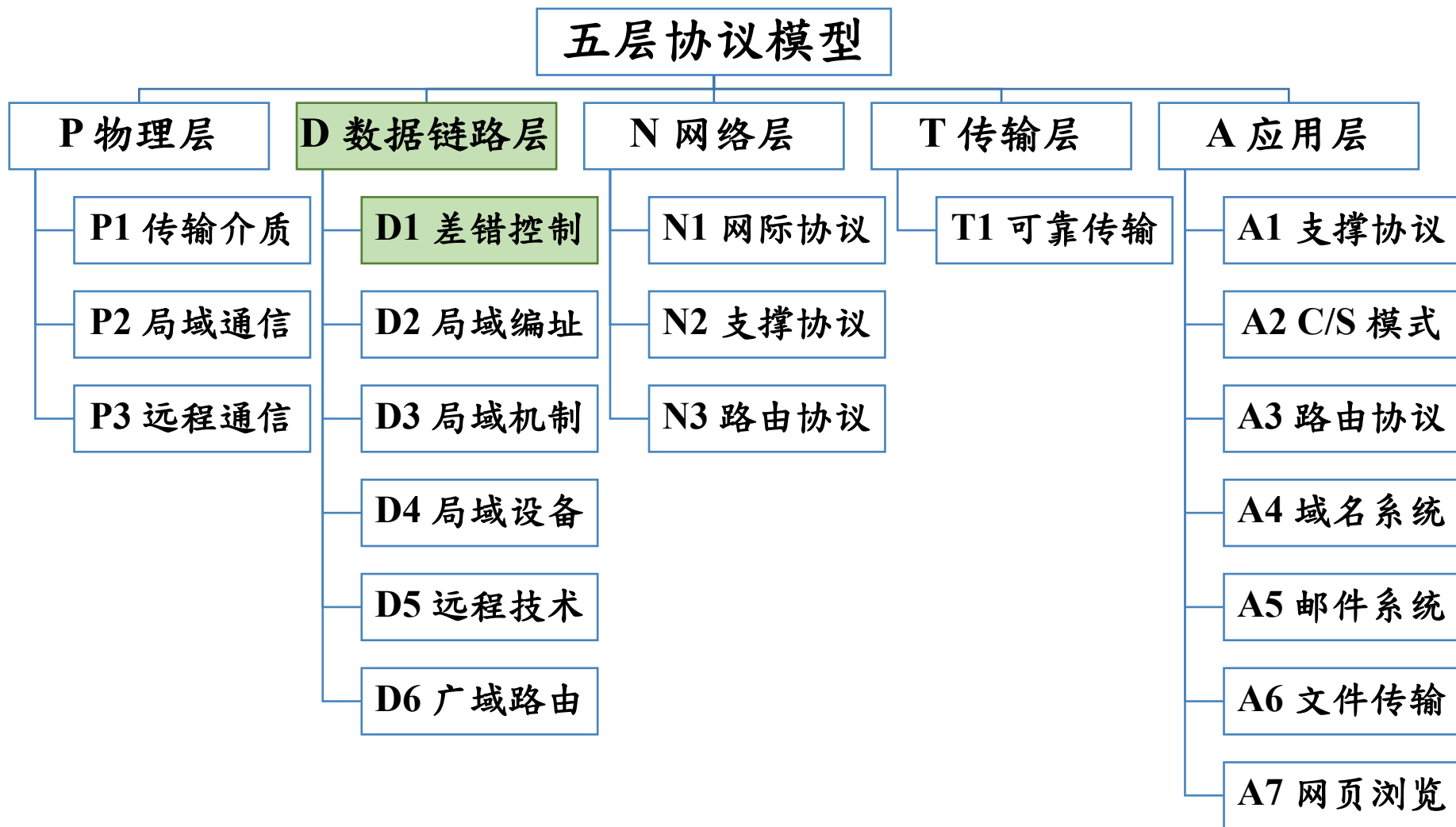


廈門大學
XIAMEN UNIVERSITY



信息学院 黄 烽
(特色化示范性软件学院) 博士, 副教授
School of Informatics Dr. Wei Huang

内容要点



主要内容

- 传输差错的原因
- 差错控制编码
 - 奇偶校验码、Internet校验和、CRC
 - 计算方法、作用与局限

对应课本章节

- **PART II Data Communication Basics**
 - **Chapter 8 Reliability And Channel Coding**



艾滋器官移植事故 台大和成大医院各罚15万

• Reactive vs. Negative



2011年8月，中国台湾《联合晚报》指出，一位邱姓男子23日不慎在家中坠楼、送到新竹南门医院被判定脑死。24日家属同意捐赠器官，成大医院也接获台大医院通知。24日傍晚台大器官移植小组进开刀房摘除器官，摘下的心肺肾等器官，火速移植给台大和成大医院5名患者，但25日，台大医院发现检验结果书面报告为阳性，以致误将艾滋感染者器官移植给病患。

报道指出，在中国台湾，现行艾滋病非属重大伤病加上保护感染者，因此不会特别注记在健保卡上，艾滋感染者若因艾滋病到医院就医时，需另外出示医疗卡。因此当意外坠楼的邱姓男子被送到医院急救、判定脑死时，从健保卡无从得知他是否有艾滋病。而邱姓男子家属没有联想到他是同性恋，属于艾滋病高危险群，错失第一道防线。

另外，目前医院没有向中国台湾卫生主管机关查询列管艾滋病患的程序，而是由医院自行检验。艾滋感染者是在新竹南门医院捐赠器官，血液检体送到台大医院检验，台大协调师出差到新竹，和台大医院检验部电话联系，不知是检验师口误，还是协调师听错，误将reactive（阳性）当成non-reactive（非阳性），发生严重错失。检验书面报告正确，只是移植团队在摘取器官进行移植时，没有依规再次检视，又错失最后防线。

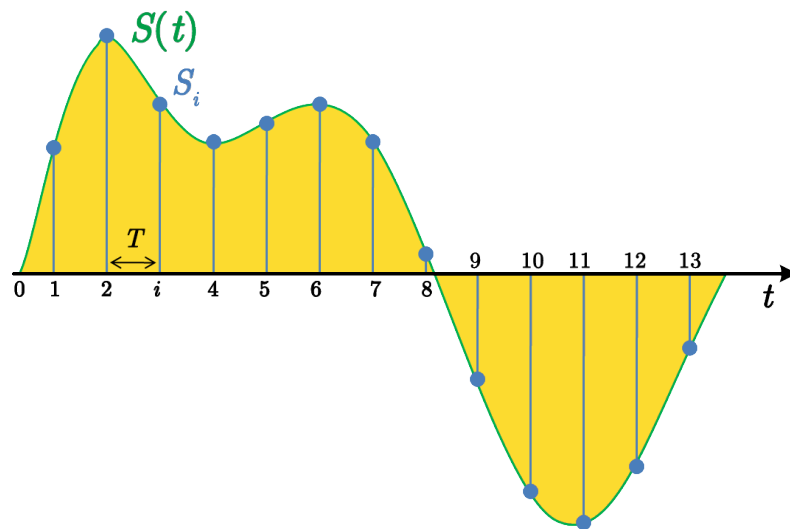
从现实社会到虚拟世界

- 现实世界：连续的

- 时间上的、数值上的，如：挥手视频，如：声波、光波
- 如果不能找到规律，则需要大量的存储能力

- 计算机世界：离散的

- 时间上：采样；数值上：编码
- 数字化
 - 1号时间、2号时间、.....
 - 1号音量、2号音量、.....



信息熵 (Entropy)

- 六个字符与六位数字的信息量

$$\begin{aligned} H(X) &= - \sum_{i=1}^n p(x_i) \log_b p(x_i) & H(X) &= - \sum_{i=1}^n p(x_i) \log_b p(x_i) \\ &= - \sum_{i=1}^{256^6} \frac{1}{256^6} \log_2 \frac{1}{256^6} & &= - \sum_{i=1}^{10^6} \frac{1}{10^6} \log_2 \frac{1}{10^6} \\ &= -256^6 \frac{1}{256^6} \log_2 \frac{1}{256^6} & &= -10^6 \frac{1}{10^6} \log_2 \frac{1}{10^6} \\ &= 6 \log_2 256 & &= 6 \log_2 10 \\ &= 48 \text{bit} & &\approx 19.93 \text{bit} \\ &= 6 \text{byte} & &\approx 2.49 \text{byte} \end{aligned}$$

目录

	1	传输差错
	2	奇偶校验码
	3	Internet校验和
	4	循环冗余检验码

传输差错 (Transmission Errors)

- 定义 (二进制) : 0变成1, 或1变成0。
- 模型
 - 突发热噪声淹没信号, 引起孤立的单比特错误
 - 原自物理过程导致突发性的连续差错
 - 因信号衰弱出现模糊差错
- 特点: 传输差错是自然产生的, 不是恶意拼凑的
 - 干扰 (Interference) : 元器件电子辐射; 宇宙背景辐射
 - 失真 (Distortion) : 长距离传输会受到干扰
 - 衰减 (Attenuation) : 通过介质的信号会变弱

降低差错率的方法

- 物理层面：从源头减少噪声

- 依据香农定理，提高信噪比可以提升信道容量
- 屏蔽线，优化天线设计，降低工作温度

- 编码层面：检测或纠正错误

- 前向错误纠正（Forward Error Correction，FEC）

- 在信息后附加用于纠错的码，接收方无需等待重传即可纠错。

- 分块错误编码（Block Error Codes）

- 将数据分块，编码依赖于给定块，和前面的块无关

- 卷积错误编码（Convolutional Error Codes）

- 连续计算编码，计算的码取决于当前输入和以前的一些位流

降低差错率的方法

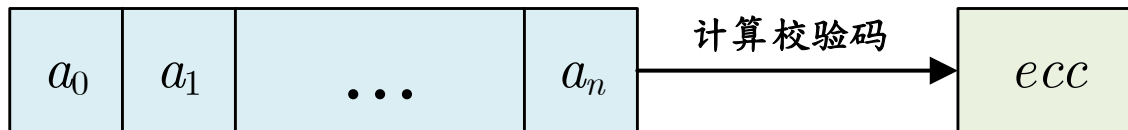
- 协议层面：协议规定出错重传机制
 - 自动重传请求 (Automatic Repeat reQuest, ARQ)
 - 每当消息到达，对方发回一个简短的确认 (ACK) 消息
 - 如果没有收到，则假设消息丢失，应重传副本
 - 接收方如果发现信息错误，则丢弃
- 错误处理是误码率和比特率之间的权衡
 - 增加冗余：部分带宽用于冗余校验，显著降低误码率
 - 减少冗余：提高有效比特率，但误码率上升

目录

1	传输差错
2	奇偶校验码
3	Internet校验和
4	循环冗余检验码

校验机制

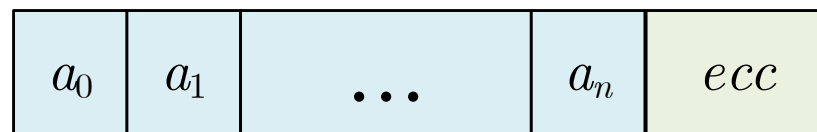
- 发送方计算校验码



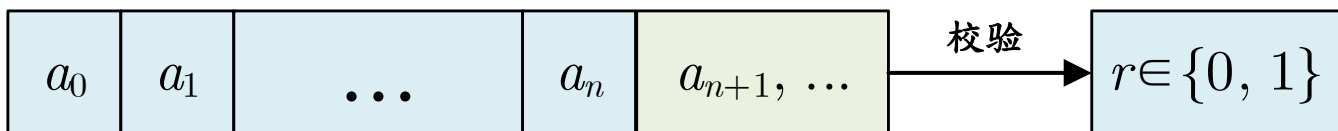
- 对于不定长的数据，可以通过padding的方式填充处理

- 发送方根据原始消息和校验码计算出待发送消息

- 一般将校验码附加在原信息后



- 接收方在收到消息后校验全部信息



- 如果校验通过，丢弃校验码，使用实际消息部分

- 如果校验不通过，尝试纠错或发起重传

校验位和奇偶校验

- 奇偶校验 (parity check) 的两种机制
 - 奇 (odd) 校验，待校验的消息应有奇数个1
 - 偶 (even) 校验，待校验的消息应有偶数个1
- 发送和接收方应有同样的模式

- 无法检测所有错误

- 如果错误位数是奇数，则可以检测到
- 否则，无法发现

模式

偶校验

发送信息

1	0	1	1	1	0	0
---	---	---	---	---	---	---

✓

消息错1位

1	0	1	0	1	0	0
---	---	---	---	---	---	---

✗

校验码错

1	0	1	1	1	0	1
---	---	---	---	---	---	---

✗

消息错2位

1	0	1	0	1	1	0
---	---	---	---	---	---	---

✓

消息错3位

0	0	1	0	1	1	0
---	---	---	---	---	---	---

✗

```

static size_t count_ones(const uint8_t msg[], size_t len) {
    size_t ones = 0;

    for (size_t i = 0; i < len; i++)
        ones += (size_t)(msg[i] & 1u);
    return ones;
}

size_t parity_gen_append(uint8_t msg[], size_t len, int is_odd) {
    size_t ones = count_ones(msg, len);

    msg[len] = (ones & 1) ^ is_odd;
    return len + 1u;
}

int parity_check(const uint8_t msg[], size_t len, int is_odd) {
    size_t ones = count_ones(msg, len);

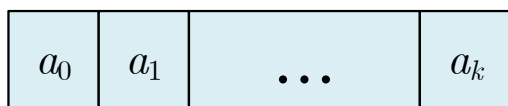
    return (ones & 1) == is_odd;
}

```

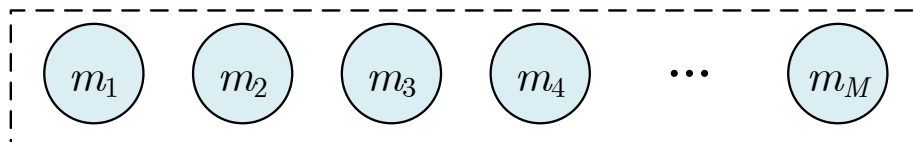
信息编码的基本概念

- 码本和码本补集的并集是所有二进制组合的集合
- 码率 (Code rate) $R = k/n$ $n = k + r$

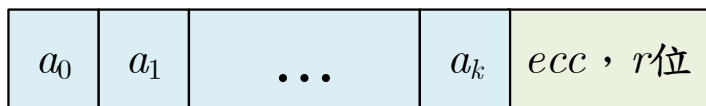
数据字 (Dataword) : 原始消息



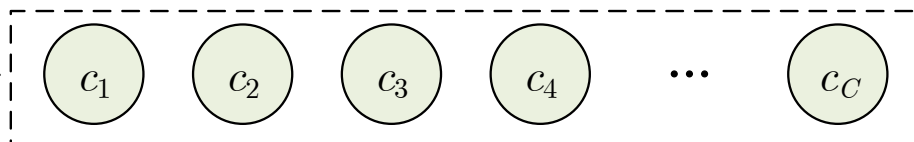
消息空间 (Message Space) : 数据字的集合



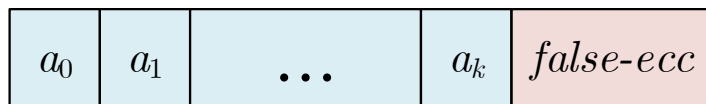
码字 (Codeword) : 数据字 + 校验码



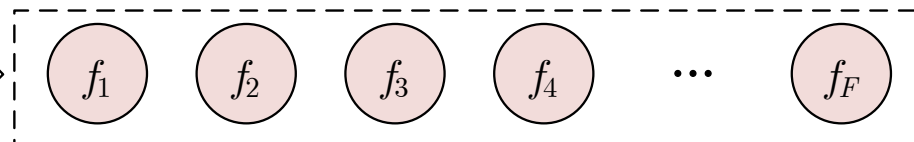
码本 (Codebook) : 所有有效码字的集合



非码字 (Non-codeword) : 无效编码

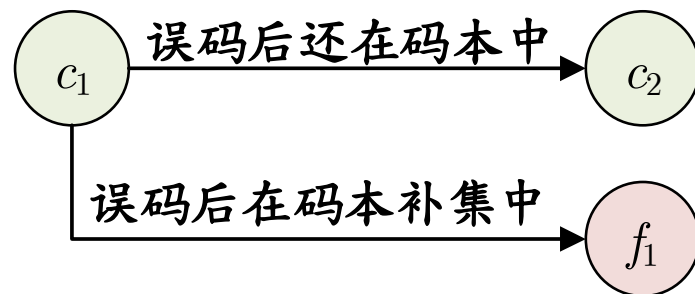


码本的补集 : 所有无效码字的集合



校验码的局限性

- 校验码存在哈希碰撞可能，导致错误无法被检测
 - 哈希碰撞：不同的数据可能产生相同的校验值
 - 没有理想的信道编码：修改足够多位可转换至合法的码字
 - 块错误码的成功关键是选择好的码本
- 无法区分出错的原始信息还是校验码
- 常规校验码仅支持校验，无法错误定位与纠正
- 无法抵御恶意篡改
- 带来额外的带宽和计算资源消耗



汉明距离：代码强度的量度

- 汉明距离：两个字符串中它们不同位的数量

```
int hamming_dist(const uint_8 a[], const uint_8 b[], size_t len) {  
    if (a == NULL || b == NULL || len == 0)  
        return -1;  
    int distance = 0;  
    for (size_t i = 0; i < len; i++)  
        distance += (a[i] ^ b[i]);  
    return distance;  
}
```

- 最小汉明距离 (minimum Hamming distance)
 - 码本中任何码字转换成另一个码字所需要改动位数的下限
 - 码本好坏的度量

纠正错误

- 奇偶校验码可以检测错误，但不能纠正错误
- 行列码扩展了奇偶校验码实现1位纠正错误的能力
 - 假设错误只有1位，则纠正1位；否则无法实现

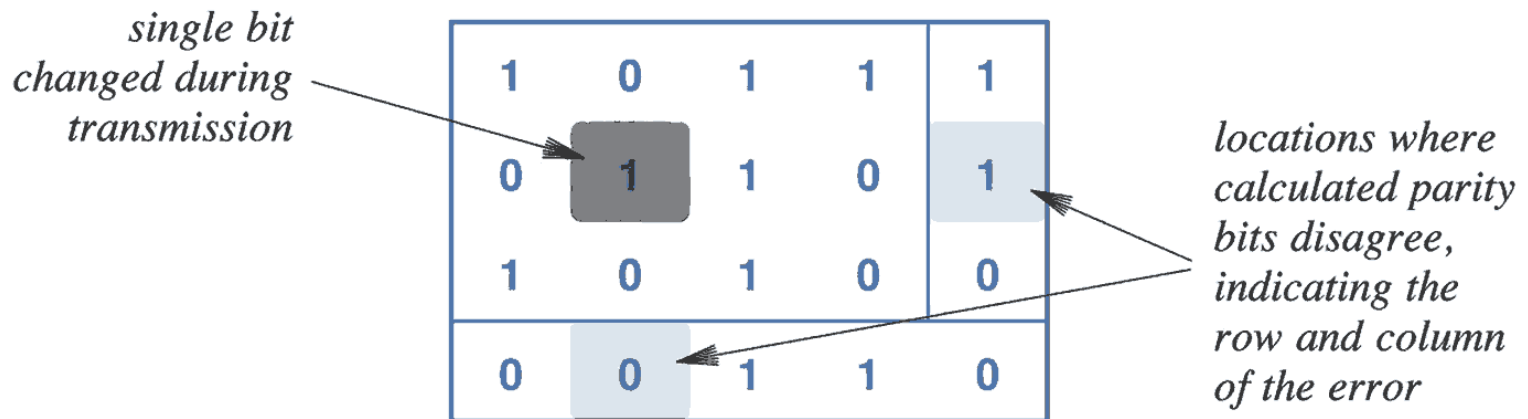


Figure 8.8 Illustration of how a single-bit error can be corrected using a row and column encoding.

目录

1	传输差错
2	奇偶校验码
3	Internet校验和
4	循环冗余检验码

Internet 校验和

- 数据字不需要固定大小，不足16位补0
- 发送者把消息每16位作为一个整数计算总和，如果结果大于16位，则重复上述过程。
- 发送方在消息后添加校验和
- 校验和尺寸小，额外传输开销小

Internet 校验和

• 发送方

H	e	l	l	o	□	w	o	r	l	d	.
48	65	6C	6C	6F	20	77	6F	72	6C	64	2E

$$4865 + 6C6C + 6F20 + 776F + 726C + 642E + \text{carry} = 71FC \xrightarrow{\text{NOT}} 8E03$$

2 71FA 2

• 接收方

48	65	6C	6C	6F	20	77	6F	72	6C	64	2E	8E	03
----	----	----	----	----	----	----	----	----	----	----	----	----	----

$$4865 + 6C6C + 6F20 + 776F + 726C + 642E + 8E03 = 271FD \longrightarrow \text{FFFF}$$

✓ 通过

计算Internet Checksum

- 以下代码源于 RFC 1071 文档。

```
/* Compute Internet Checksum for "count" bytes beginning at
location "addr". */
register long sum = 0;
while (count > 1)
    sum += *(unsigned short *) addr++;  count -= 2;
if (count > 0) /* Add left-over byte, if any */
    sum += *(unsigned char *) addr;
while (sum >> 16) /* Fold 32-bit sum to 16 bits */
    sum = (sum & 0xffff) + (sum >> 16);
checksum = ~sum;
```

目录

1	传输差错
2	奇偶校验码
3	Internet校验和
4	循环冗余检验码

循环冗余检验码 (CRC)

- 循环冗余检验码 (Cyclic Redundancy Codes)

- 一种根据网络数据包或文件等数据产生简短固定位数校验码的散列 (Hash) 函数，主要用来检测或校验数据传输或者保存后可能出现的错误。

- 重要特征

- 任意消息字长

- 出色的错误检测

- 快速硬件实现 (位移寄存器，异或门)

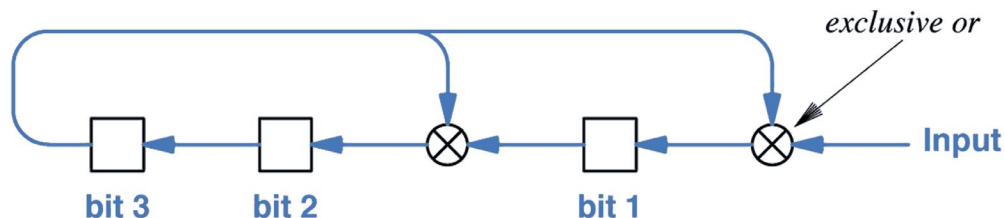


Figure 8.13 A hardware unit to compute a 3-bit CRC for $x^3 + x^1 + 1$.

循环冗余检验码 (CRC)

- 可以视为不带借位的二进制数除法
 - 每个位的二进制数作为一个多项式的系数
- 除数多项式称为生成多项式 (generator polynomial)
 - 一个理想的多项式是不可约的 (只能被自己和1整除)
 - 有一个以上的非零系数多项式可检测所有的单比特错误

除数1011可视为 $x^3 + x + 1$

被除数1010000可视为 $x^6 + x^4$

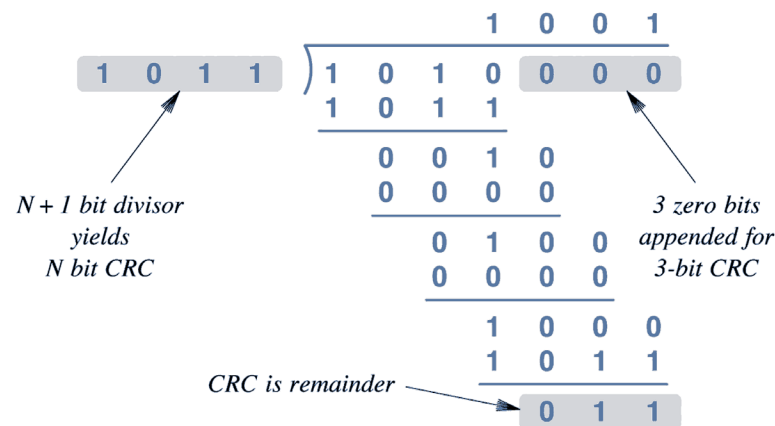


Figure 8.12 Illustration of a CRC computation viewed as the remainder of a binary division with no carries.

生成多项式的国际标准

• 生成多项式 $G(x)$ 的国际标准

标准	生成多项式
CRC-8	$x^8 + x^2 + x + 1$
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$
CRC-12	$x^{12} + x^{11} + x^3 + x + 1$
CRC-16	$x^{16} + x^{15} + x^2 + 1$
CRC-CCITT	$x^{16} + x^{12} + x^5 + 1$
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

```
// CRC-8 多项式:  $x^8 + x^2 + x^1 + 1 = 0x07$ 
```

```
#define CRC8_POLY 0x07
```

```
#define CRC8_INIT 0x00
```

```
uint8_t crc8_byte(uint8_t data, uint8_t crc) {  
    crc ^= data;  
    for (int i = 0; i < 8; i++)  
        if (crc & 0x80)  
            crc = (crc << 1) ^ CRC8_POLY;  
        else  
            crc = crc << 1;  
    return crc;  
}
```

```
uint8_t crc8(const uint8_t data[], int len) {  
    uint8_t crc = CRC8_INIT;  
    for (int i = 0; i < len; i++)  
        crc = crc8_byte(data[i], crc);  
    return crc;  
}
```

帧格式和错误检测机制

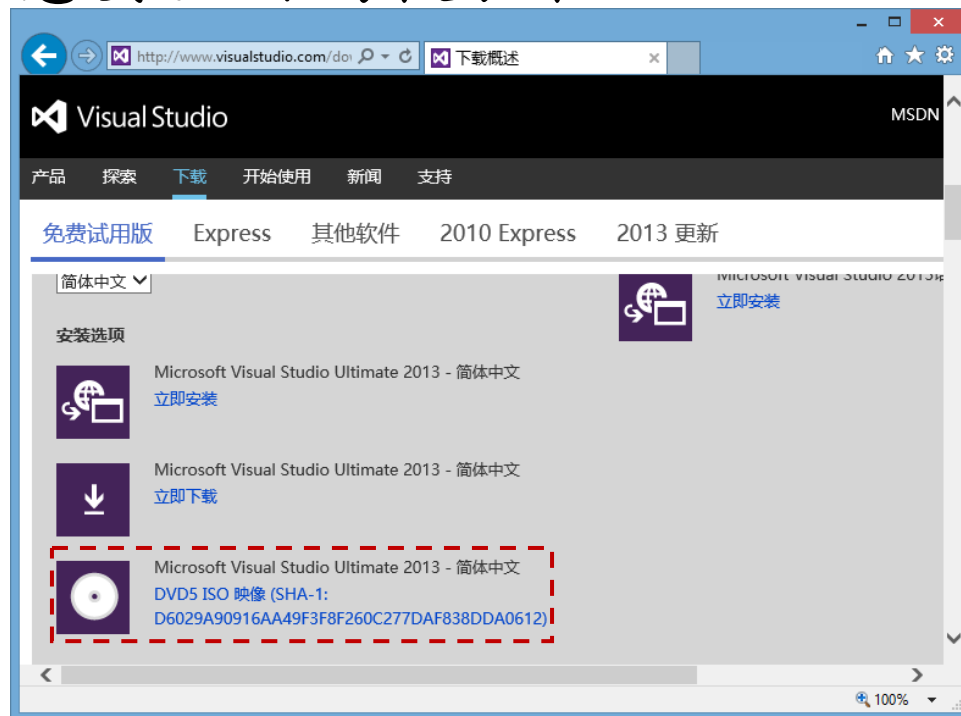
- 错误检测信息作为帧的一部分传输

帧首定界符	包含字节填充的数据块	帧尾定界符	CRC
-------	------------	-------	-----

- 发送方计算信息的校验，随着帧数据发送附加信息。
- 接收方用相同的方法计算校验值
 - 一般将校验值一起计算，而不是计算校验值再与发来的校验值比较。
- 一般构造复杂的发送方，简单的接收方。

更复杂的校验码

- 所有错误检测方案都有局限性
 - 传输错误可能以特定模式发生，恰好避开检测机制。
- 恶意的消息篡改应使用信息安全用的校验和
 - MD5、SHA系列



谢谢观看



厦門大學
XIAMEN UNIVERSITY



信息学院 黄 烽
(特色化示范性软件学院) 博士, 副教授
School of Informatics Dr. Wei Huang